

La manutenzione

La manutenzione del software viene definita formalmente come il processo di modifica di un prodotto software successivo al suo rilascio in esercizio.

L'impatto di queste modifiche non è sempre uguale e si può dividere in due macro-categorie:

- **Ordinario:** comprende le modifiche di routine e la gestione quotidiana del software.
- **Straordinario:** riguarda eventi di portata massiccia, come lo fu il problema dell'Anno 2000 (Millennium Bug), l'introduzione dell'Euro o la migrazione verso nuovi ambienti e linguaggi di programmazione. Questi interventi sono talmente impattanti che richiedono uno sprint dedicato, se non addirittura l'apertura di un progetto ex novo.

Entrando nel dettaglio tecnico, l'ingegneria del software classifica le attività di manutenzione in tre tipologie specifiche:

1. **Manutenzione Correttiva:** È focalizzata in modo mirato sull'eliminazione dei bug e dei malfunzionamenti del sistema.
2. **Manutenzione Adeguativa:** Si rende necessaria quando il software deve adattarsi a dei cambiamenti sopravvenuti nel suo ambiente. Questa si ramifica ulteriormente in:
 - *Adeguamento Operativo:* legato a modifiche dell'hardware o del software di base, come ad esempio il porting su un nuovo sistema operativo.
 - *Adeguamento Normativo:* legato a cause esterne, come l'entrata in vigore di nuove leggi o l'introduzione di una nuova valuta.
3. **Manutenzione Evolutiva:** Riguarda tutti quegli interventi volti ad arricchire gli applicativi esistenti introducendo nuove funzionalità o migliorando caratteristiche non funzionali. È importante sottolineare che in questa categoria rientrano anche i miglioramenti strutturali volti a ridurre il debito tecnico, operazione nota come *refactoring*.

Il debito tecnico

Viene associato il "debito tecnico" come una metafora molto efficace per rappresentare la perdita di produttività causata dall'adozione di soluzioni di sviluppo a "corto respiro". La regola d'oro è che il costo dei cambiamenti tende a crescere nel tempo; l'unico modo per invertire questa tendenza è "ripagare" il debito accumulato tramite un'adeguata attività di refactoring.

Questo scenario pone chi sviluppa e gestisce il software di fronte a un vero e proprio "dilemma della manutenzione", diviso tra due approcci opposti:

- **La via veloce (Accumulo di debito):**

Si sceglie di applicare la modifica per risolvere il problema nel modo più rapido possibile. Sebbene sia una scelta economicamente vantaggiosa nel breve termine, essa tende ad aumentare la confusione nel codice, rendendo il software progressivamente meno comprensibile e molto più ostico da modificare in futuro.

- **La via virtuosa (Senza debito):**

Si decide di applicare la modifica risolvendo il problema alla radice, senza accumulare ulteriore debito tecnico. Questo approccio risulta chiaramente più costoso e lento nell'immediato, ma i suoi frutti vengono ripagati nel lungo termine, poiché permette di preservare intatta la comprensibilità e la naturale modificabilità del software.

La visione naturalistica e le leggi di Lehman

Nel 1976, i ricercatori Lehman e Belady formularono delle leggi fondamentali diventate pietre miliari nello studio dell'evoluzione del software.

- **I Legge (Cambiamento Continuo):** Un programma applicato a un contesto del mondo reale deve subire modifiche costanti, altrimenti rischia di diventare progressivamente meno utile per gli utenti. Questo ciclo continuo di alterazioni si interrompe soltanto quando si stabilisce che è economicamente più conveniente sostituire l'intero programma con un sistema completamente nuovo.
- **II Legge (Entropia Crescente):** Ogni volta che un software viene modificato, la sua complessità strutturale cresce in modo naturale, provocando un degrado della sua architettura originale. L'unico modo per contrastare questo fenomeno (ovvero per ridurre l'entropia) è compiere del lavoro aggiuntivo specifico volto a mitigare o risolvere tale complessità, che si traduce nella pratica del refactoring.

Ciclo di vita di un software

Diverse fasi di vita di un software:

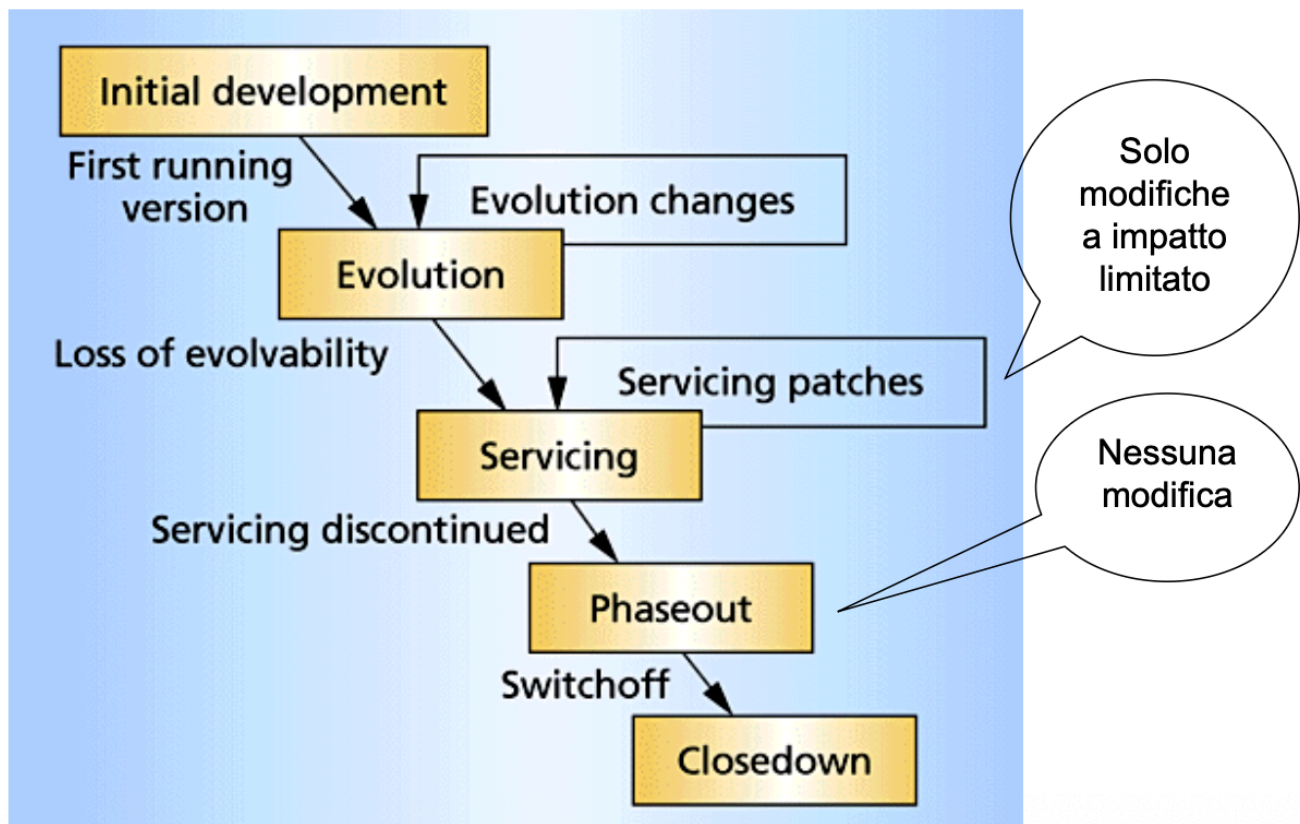
1. **Initial Development (Sviluppo Iniziale):** La fase in cui il software viene progettato e implementato da zero, portando al rilascio della prima versione funzionante (*First running version*).
2. **Evolution (Evoluzione):** Il sistema subisce costanti modifiche evolutive per soddisfare i bisogni degli utenti e rispondere ai mutamenti ambientali. Con il passare del tempo e l'accumulo di modifiche, si assiste però a una progressiva perdita di evolvibilità (*Loss of evolvability*), che rende sempre più difficile applicare modifiche strutturali importanti senza destabilizzare il sistema.

3. **Servicing (Assistenza/Manutenzione Minore):** A causa della perdita di evolvibilità, il software entra in una fase in cui non si introducono più nuove grandi funzionalità. Ci si limita esclusivamente ad applicare correzioni mirate e patch a basso impatto per mantenere il sistema operativo e sicuro.
4. **Phaseout (Fase di Ritiro):** Quando anche l'assistenza viene interrotta (*Servicing discontinued*), il software entra in uno stato in cui non viene applicata più alcuna modifica. Il programma resta operativo così com'è mentre gli utenti vengono gradualmente migrati verso altre soluzioni.
5. **Closedown (Chiusura/Dismissione):** Si giunge allo spegnimento definitivo (*Switchoff*) del sistema, che cessa totalmente di essere utilizzato.

Gestione dei prodotti versionati

Nello scenario reale dei prodotti software commerciali, queste fasi non si applicano in modo lineare a tutto il codice, ma vengono gestite in parallelo attraverso il versionamento (es. versione 1.0,1.1,2.0).

Mentre una specifica versione (ad esempio la 1.0) si trova nella sua fase di *Servicing* o di *Phaseout*, il team di sviluppo lavora in parallelo all'*Evolution* di una nuova versione major (la 2.0), garantendo la continuità aziendale e il progressivo trasferimento degli utenti da un ciclo di vita all'altro.



- **Initial Development & Evolution:** Il ciclo inizia con lo sviluppo iniziale che porta alla "First running version" (prima versione funzionante). Da qui il software entra nel loop dell'**Evolution**, dove subisce continui "Evolution changes" (modifiche evolutive che aggiungono funzionalità e valore).
- **Il punto critico (Loss of evolvability):** Arriva un momento in cui la struttura del software è troppo degradata o complessa. Avviene la *perdita di evolvibilità*. Questo è un punto di non ritorno: il software transita allo stato successivo.
- **Servicing:** In questa fase il software non riceve più nuove funzionalità. Si entra nel loop delle "Servicing patches". Come specifica il balloon nell'immagine, qui sono ammesse **"Solo modifiche a impatto limitato"** (es. bug fix critici o patch di sicurezza) per non rischiare di rompere il sistema.
- **Phaseout & Closedown:** Quando si decide che non vale più la pena mantenere il codice ("Servicing discontinued"), si passa al **Phaseout**. Il balloon chiarisce che in questa fase c'è **"Nessuna modifica"**: il software continua a funzionare ma è abbandonato a se stesso, in attesa che gli utenti migrino altrove. Infine, lo "Switchoff" (lo spegnimento dei server o il blocco delle licenze) porta al **Closedown** definitivo.

Evoluzione di prodotti software versionati

<version>.<release>

1.0

1.1

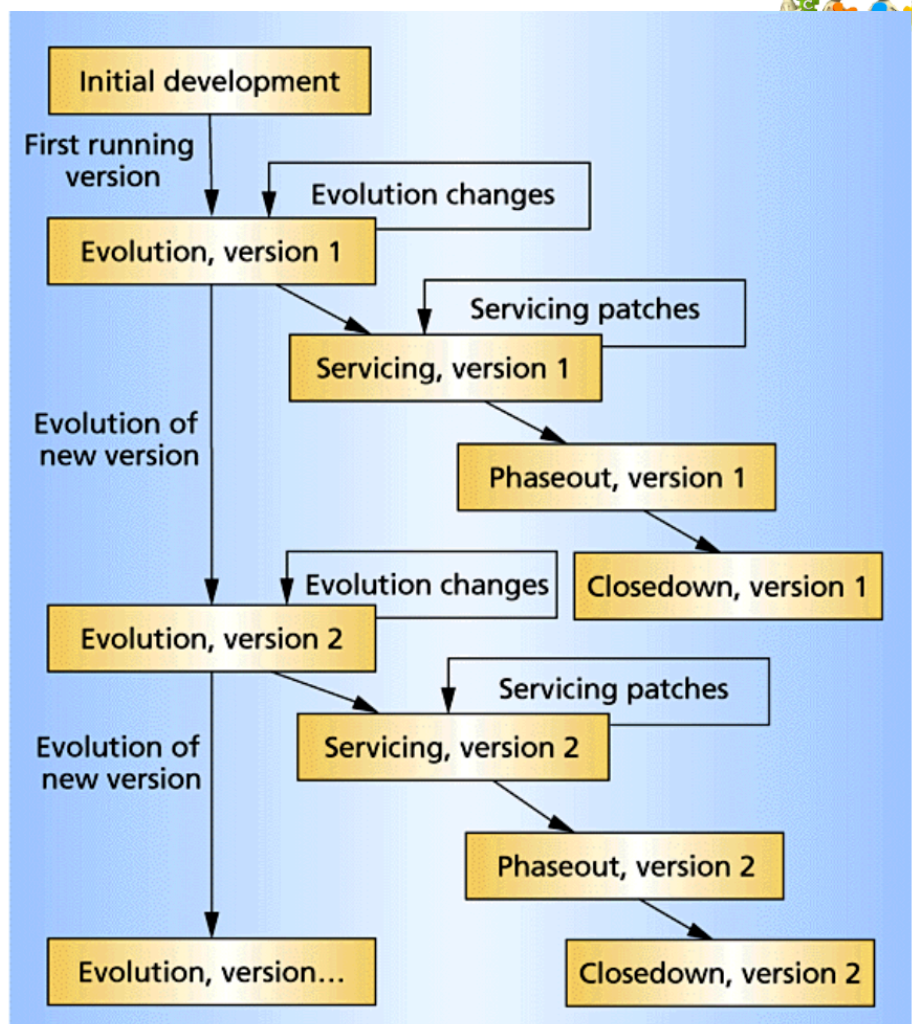
1.2

2.0

2.1

...

...



- **Sviluppo Parallelo e Sovrapposizione:** Mentre la Versione 1.0 affronta i suoi "Evolution changes" diventando 1.1 e poi 1.2, a un certo punto inizia in parallelo l'"Evolution of new version" (lo sviluppo della Versione 2).
- **Slittamento di Fase:** Nel momento in cui la Versione 2 diventa pronta e inizia il suo ciclo di "Evolution", la vecchia Versione 1 viene declassata. La Versione 1 entra quindi nella fase di **Servicing** (ricevendo solo patch), per poi scivolare in **Phaseout** e infine in **Closedown**.
- **Ciclo Infinito:** Questo schema a cascata sfalsata si ripete indefinitamente (Versione 2, Versione 3, ecc.). Garantisce che l'azienda abbia sempre un prodotto nella fase altamente redditizia di "Evolution", mentre gestisce dolcemente l'obsolescenza e l'abbandono delle versioni più vecchie per cui si è verificata la "Loss of evolvability".